

Exercise 07 – IT24101516

1).

Program	Main collective	Memory allocation	Manual sum at root?	Result available to
Exercise 1	MPI_Bcast + Send/Recv	Every process has full array	Yes	Root
Exercise 2	MPI_Scatter + Send/Recv	Root: full array; others: chunk	Yes	Root
Exercise 3	MPI_Scatter + MPI_Gather	Root: full array + all sums; others: chunk	Yes	Root
Exercise 4	MPI_Scatter + MPI_Reduce	Root: full array; others: chunk	No	Root
Exercise 5	MPI_Scatter + MPI_Allreduce	Root: full array; others: chunk	No	Every process
Exercise 6	MPI_Scatter + MPI_Scan	Root: full array; others: chunk	No	Prefix sum for every process

2).

Program	2 Processes (s)	4 Processes (s)	8 Processes (s)
Bcast + Send/Recv	0.002401	0.006583	0.014190
Scatter + Send/Recv	0.003148	0.003745	0.004417
Scatter + Gather	0.003136	0.004215	0.005346
Scatter + Reduce	0.003139	0.004523	0.004926
Scatter + Allreduce	0.002632	0.003438	0.005388
Scatter + Scan	0.004259	0.003053	0.005503

All six programs gave the total sum 500,000,500,000 correctly. When we use 2 processes Bcast+Send/Recv gives the best timing whereas Scatter+Scan gives the best timing using 4 processes whereas Scatter+Send/Recv gives the best timing using 8 processes.

Using additional processes doesn't guarantee better efficiency due to the overheads involved in the communication and synchronization in MPI. The Bcast program was particularly inefficient in case of higher number of processes as compared to others since all of the array elements were transmitted by using the bcast method.

3). It is recommended to use MPI_Scan instead of MPI_Allreduce if each node needs to compute the cumulative value until itself. Consider an example where we calculate the prefix sum in a distributed manner, each node requires the sum computed so far by itself and other previous nodes. MPI_Scan gives us that directly, whereas MPI_Allreduce gives each node the same final sum.